ALGORITHMS AND COMPLEXITY

1. a) Give a tight bound for each of the following recurrence relations, or explain why it's not possible to do so.

Carefully justify your answers.

i) $T(n) = T(n/3) + T(n/3) + T(n/3) + 1$ [3]

Answer:
$d = 0, a = 3, b = 3$. So $\log_b a = \log_3 3 = 1 > 0 = d$ and the complexity is $O(n^{\log_b a}) = O(n)$.
Almost all students solved this correctly, although a few didn't realise that $1 = O(n^0)$ and put $d = 1$.

ii) $T(n) = 9T(n/3) + 3n^2$ [3]

Answer:
$d = 2, a = 9, b = 3$. So $\log_b a = \log_3 9 = 2 = d$ and the complexity is $O(n^d \log n) = O(n^2 \log n)$.
Most students solved this correctly, although some wrote $T(n) = 3(3T(n/3) + n^2)$ and then solved the problem for $a = b = 3$, $d = 2$.

iii) $T(n) = T((n/2)^2)/2 + 1/n$ [4]

Answer:
Firstly, this recurrence does not fit the conditions of the Master Theorem. Secondly, it cannot give a well-defined bound as $(n/2)^2 > n$. So, for example $T(8) = T(16)/2 + 1/8$, $T(16) = T(64)/2 + 1/16$, etc.
Many students correctly realised that the Master Theorem didn't apply, but the majority did not note that the recurrence is not well-defined.

b) Give a bound on the complexity of the following operations, and justify your answers in terms of a specific algorithm. *The algorithm code does not need to be given, unless it is needed to support your argument. You do not have to give the best possible algorithm.*

i) Multiplication of two $n \times n$ matrices. [4]

Answer:
If $M = A \cdot B$ then $M_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$. Computing each item M_{ij} takes $\Theta(n)$ operations and there are n^2 such operations so the total number of operations is $\Theta(n^3)$. This gives an upper bound of $O(n^3)$ on matrix multiplication.
Most students knew the result, but some couldn't explain why. $O(n^2)$ and $O(n^4)$ were quite common answers. A few students noted that in addition to the naive $O(n^3)$ solution there were better solutions using divide and conquer, but didn't give details.

ii) Sorting an array of length n. [6]

Answer:
One option is merge sort, which is a form of divide and conquer. Divide the array into two parts of length $n/2$. Conquer these by recursion. Combine by iterating simultaneously over the subarrays, taking the smaller of the two parts at each stage. If total complexity is $T(n)$ then each subarray will have $T(n/2)$ steps and the combination step is $O(n)$

Answers

so $T(n) = 2T(n/2) + O(n)$. Applying the master theorem gives $a = b = 2, d = 1$ so $\log_b a = \log_2 2 = 1 = d$ and therefore $T(n) = O(n \log n)$. Most students knew the result $O(n \log n)$ and the majority were able to justify this to some extent as a divide and conquer merge sort. A few chose insertion or bubble sort algorithms instead (which was fine). A few also showed the minimum bound for comparison based sort.

Master Theorem. If $T(n)$ satisfies

$$T(n) = a\,T(\,n/b\,) \;+\; O(n^d)$$

for some $a > 0$, $b > 1$ and $d \geq 0$, then

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

Answers

2. A univariate polynomial p in variable x consists of a sum of n_p terms:

$$p = \sum_{i=1}^{n_p} c_p[i] x^{d_p[i]}$$

where $c_p[1], \ldots, c_p[n]$ are coefficients and $d_p[1], \ldots, d_p[n]$ are integer degrees with $d_p[i] \geq 0$.

You may assume that any arithmetical operation is $O(1)$ and that appending to a vector is $O(1)$.

a) Multiplication of two polynomials p and q is defined as:

$$p \times q = \sum_{i=1}^{n_p} \sum_{j=1}^{n_q} c_p[i] \cdot c_q[j] \cdot x^{d_p[i]+d_q[i]}$$

i) What is the time complexity of polynomial multiplication based on the formula above? [3]

Answer:
The two sums become two nested loops with lengths n_p and n_q. Each iteration does a fixed number of arithmetic operations and appends an item so is $O(1)$. Therefore, the total time complexity is $O(n_p n_q)$.
Most students solved this. Some put $O(n^2)$ as they were not familiar with the idea of complexity in two variables. Quite a few students didn't write any justification of the result. A few students assumed that they were calculating the value for some given value of x rather than a representation of the polynomial, but this didn't affect the complexity as they assumed the power operation was $O(1)$.

ii) Using this algorithm, what is the best achievable time complexity for calculating p^k for an arbitrary polynomial p and k a power of 2 ($k = 2, 4, 8, 16, \ldots$)? [7]

Answer:
We can form p^2 with complexity $O(n_p^2)$ and the result has length n_p^2. Then we form $p^4 = (p^2)^2$, $p^8 = (p^4)^2$, etc.

Let $N_i = n_p^{2^i}$, then $q_i = p^{2^i}$ has length N_i and the squaring operation $q_i = q_{i-1}^2$ has time complexity $O(N_i)$. So the total time complexity is

$$\begin{aligned}
\sum_{i=1}^{\log_2 k} O(N_i) &= O\left(\sum_{i=1}^{\log_2 k} N_i\right) \\
&= O(\overbrace{n_p^2 + n_p^4 + \cdots + n_p^{k/2}}^{\log_2 k - 1 \text{ terms each smaller than } n_p^{k/2}} + n_p^k) \\
&= O((\log_2 k - 1) n_p^{k/2} + n_p^k) \\
&= O(n_p^k)
\end{aligned}$$

The most common solution was $O(n^2 \log k)$ which is what you get if you use repeated squaring but don't consider that the number of terms changes at each stage. Some didn't use repeated squaring but did consider the change in the number of terms, and got a result of $O(n^{k+1})$,

Answers

or (incorrectly) $O(n^k)$ if they just gave the complexity of the final multiplication. A few didn't consider the change in number of terms or use repeated squaring and got $O(n^2k)$. A reasonable number used the direct formula $\sum_{i_1=1}^{n}\sum_{i_2=1}^{n}\cdots\sum_{i_k=1}^{n} c[i_1]c[i_2]\cdots c[i_k]x^{d[i_1]+\cdots+d[i_k]}$ and concluded that the complexity was either $O(n^k k)$ or (incorrectly) $O(n^k)$. A reasonable number of students also considered the change in length, but assumed that p^k would have have length kn_p (which is the degree, and would be true for canonical polynomials). Quite a few students used a form of "inductive guessing" concluding from the fact that p^2 was $O(n_p^2)$ that p^k would be $O(n_p^k)$, but without giving any clear justification.

b) A non-zero polynomial is *canonical* if $d_p[i] = i-1$ and $c_p[n_p] \neq 0$. This ensures that all powers of x must be present, they are ordered from smallest to largest power, there are no repeated terms, and the degree of the polynomial is $n_p - 1$.

Students found it difficult to unpack this mathematical definition.

If a polynomial is not known to be canonical, we call it *irregular*.

i) Give pseudocode for converting an irregular polynomial p to a canonical polynomial, and state the time complexity. You may rely on common library operations on vectors. [6]

Answer:
Python code:

```
def canonical(coeffs, degrees):
    n = max(degrees) # O(n_p)
    canonical_coeffs = zeros(n+1, dtype=int) # O(1)
    for i in range(len(coeffs)):             # n_p times
        canonical_coeffs[degrees[i]] += coeffs[i] # O(1)
    return canonical_coeffs
```

Total cost is $O(n_p)$.
Many students didn't understand the definition. Some thought it was enough to order the polynomial in terms of powers. Some considered adding missing terms but not combining repeated terms, and vice versa. Quite a few thought that you needed all coefficients to be nonzero, and would therefore raise an error if terms were missing or add terms with random coefficients! Very few students got the efficient solution above. The most common involved sorting by degree first giving $O(n\log n)$ complexity. In analysing the complexity, many students would use insertion into a vector and assume it was $O(1)$ (or even not include it at all). Also note that there is a slight error in the answer above that a very few students picked up on. The complexity should be $O(\max(n_p, \max_i(d_p[i])))$ rather than $O(n_p)$ (e.g. if the polynomial was $p(x) = 1 + x^{1000}$ it would be 1000 operations not 2 because you need to initialise the coefficient list to zero).

ii) If we have two canonical polynomials p and q and want to produce a canonical result $p \times q$, what is the time complexity? You do not need to give pseudo-code, but justify your result. [4]
Answer:

Answers

We still need to compute $c_p[i]c_q[j]$ for all $1 \le i \le n_p$, $1 \le j \le n_q$ so the algorithm must be $\Omega(n_p n_q)$, and from the earlier part the result is $O(n_p n_q)$ therefore it must be $\Theta(n_p n_q)$.
Most students simply wrote that it was the same as for irregular polynomials. Many got an inefficient result here because they assumed that you should compute using irregular, and then apply the previous part to simplify to canonical. A couple of students improved on the answer above by noting that multiplication of polynomials is equivalent to convolution and can therefore be done as FFT+multiply+IFFT at cost of $O(n \log n)$.

iii) What is the time complexity of calculating the canonical result p^k where p is canonical and k is a power of 2? [6]

Answer:
Calculating p^2 requires $O(n_p^2)$ operations but the result has length $2n_p$ instead of n_p^2. So p^4 requires $(2n_p)^2$ operations, p^8 requires $(4n_p)^2$ operations and so on. Therefore the total complexity is

$$\begin{aligned}\sum_{i=1}^{\log_2 k} O(2^{2i} n_p^2) &= O\left(n_p^2 \sum_{i=1}^{\log_2 k} 2^{2i}\right) \\ &= O(n_p^2 2^{2\log_2 k}) \\ &= O(n_p^2 k^2)\end{aligned}$$

Most students repeated their answer for irregular polynomials since they hadn't taken the increasing length into account.

iv) Is it better to calculate p^k using irregular or canonical polynomials? [4]

Answer:
The complexity for irregular polynomials is $O(n_p^k)$ and for canonical it is $O(k^2 n_p^2)$. If $n_p > 1$, the irregular method is exponential in k whereas the canonical method is polynomial in k, therefore the canonical method is better.
Many students concluded that canonical polynomials were better than irregular, even if their complexity analysis showed they were the same or worse.

Overall, the majority of students did well on the first question, but found the second question challenging.

Answers